



Introduction to Unity

By Antoine McNabb-Baltar



Lecture plan

- What is a game engine
- Game engine typical structure
- Getting started with the Unity editor.
- What is a scene graph ?
- Why use a scene graph ?
- Some notion of linear algebra
- Unity Scene graph
- GameObjects
- Transforms
- Components,
- Adding gameplay
- Unity .NET API overview & important classes.

The Unity Engine

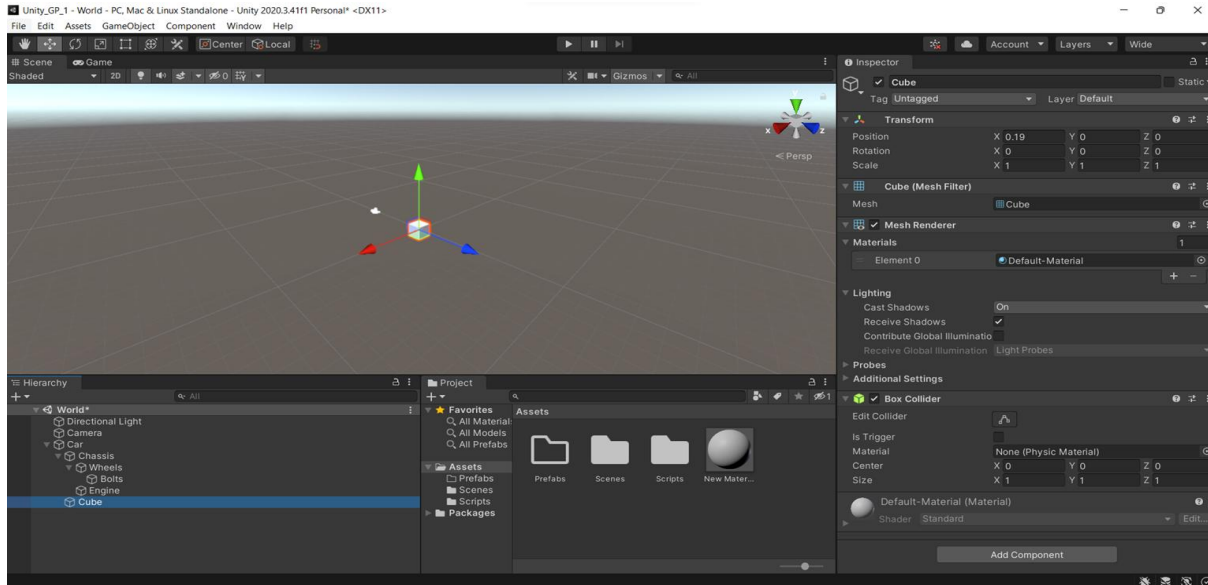


What is a game engine ?

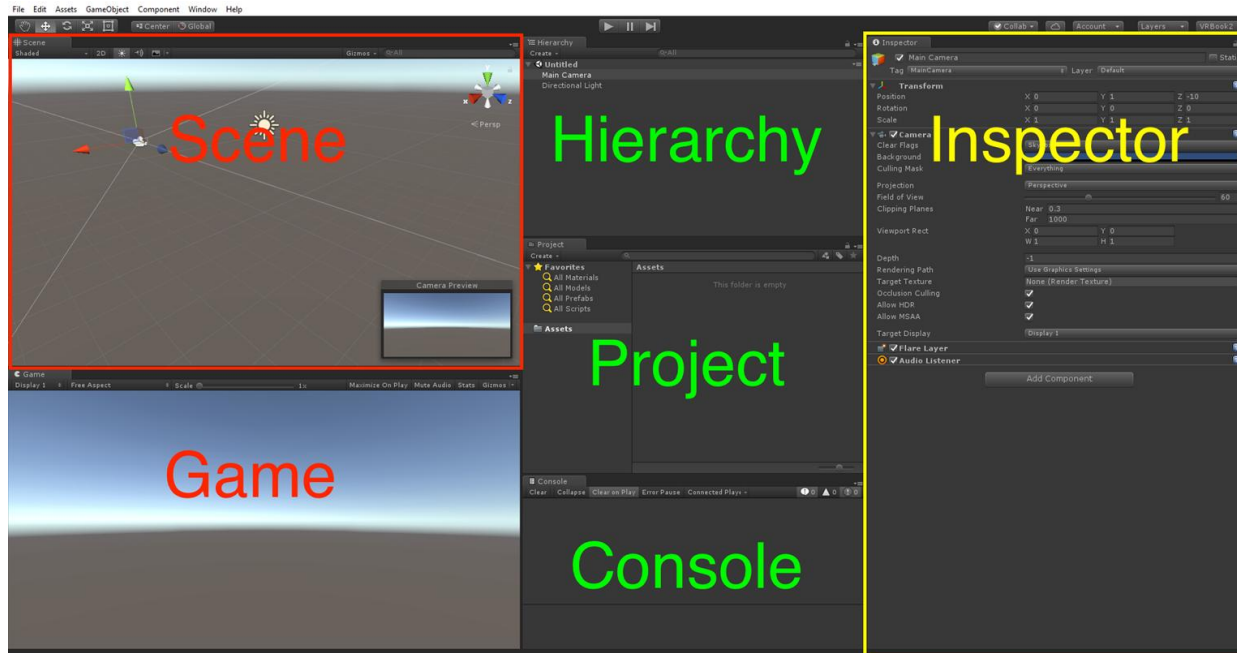
A game engine is a **set of tools** used to simplify the process of making games. They usually include a list of functionality that are used in practically all games :

- A rendering engine ("renderer") for 2D or 3D graphics
- A physics engine for managing movement and collision detection ,
- A sound engine to manage sound in 2D or 3D space
- Some scripting tool to build interactivity
- An animation framework to allow the playing of animation
- An artificial intelligence framework to build NPC behaviours and make them navigate your levels
- Networking tools to build multiplayer games
- Memory management and monitoring tools
- A scene graph to manage objects in virtual space

Getting started with the Editor



Getting started with the Editor

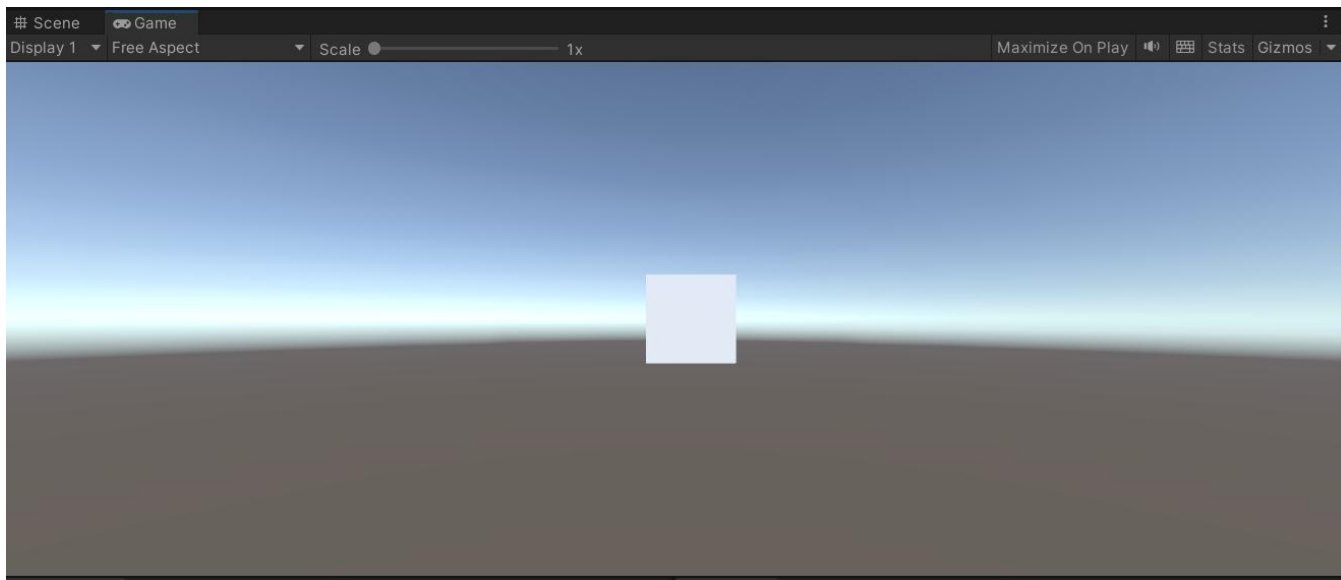


Editor controls



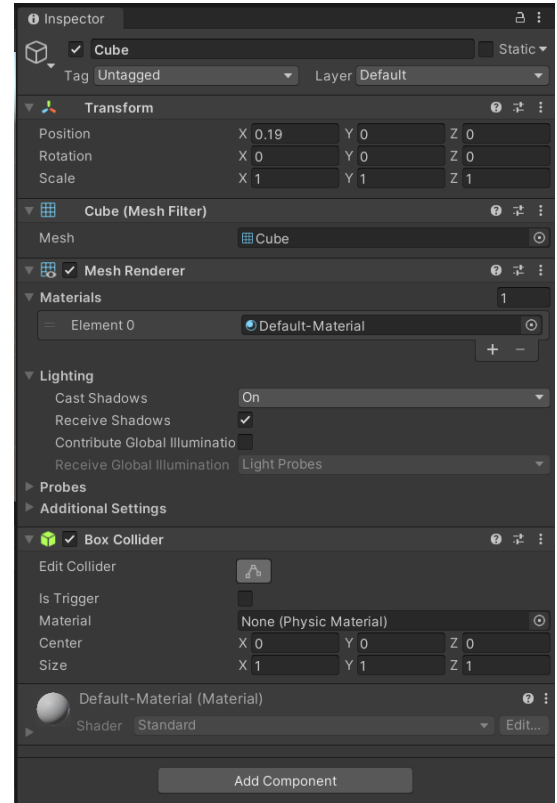
- The **Hand** tool allows you to move the engine viewport in your world.
- The **move** tool allows you to move a selected GameObject.
- The **rotate** tool allows you to rotate a selected GameObject.
- The **scale** tool allows you to modify the scale of a selected GameObject .

The game viewport (What your game actually sees)



The inspector

The Inspector displays detailed information about your currently selected **GameObject**, including all attached **Components** and their **properties**.



The project window.

Shows the Asset folder hierarchy of your project .

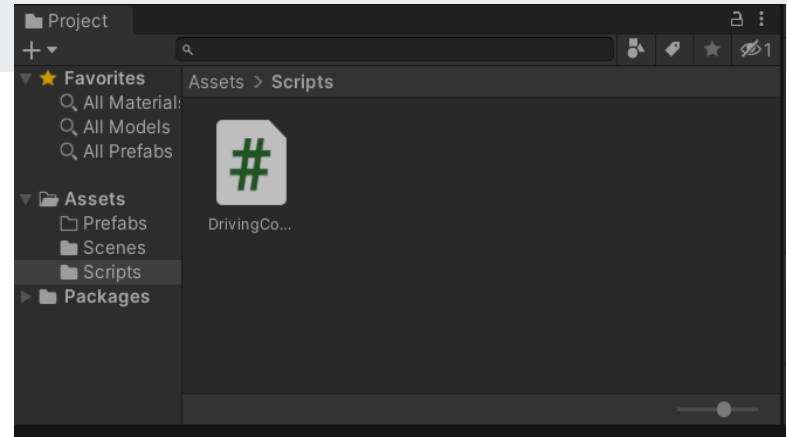
This contain all the files you create and generate for your game as well as any scene you build.

A common guideline for your project setup is to separate files and script by their uses.

So all art assets , coding assets , scenes , prefabs and materials are in separate folders

Here is a few general folder setup in a Unity project :

- Project Folder
 - Assets
 - Materials
 - Texture
 - Scene
 - Prefabs
 - Scripts



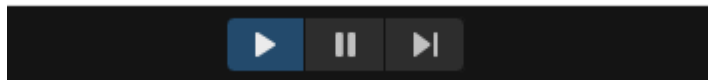


The play controls

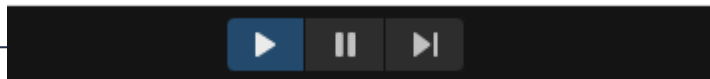
When your ready to test your game use the play button on the top of the interface.

The pause button allows you to pause your game mid execution

The button on the far left allows you to play your game frame by frame once it is paused.



Playing your game



Play

Enters Play Mode and runs your game
live inside the Editor



Pause

Freezes execution mid-game so you can
inspect state



Step

Advances the paused game exactly one
frame at a time

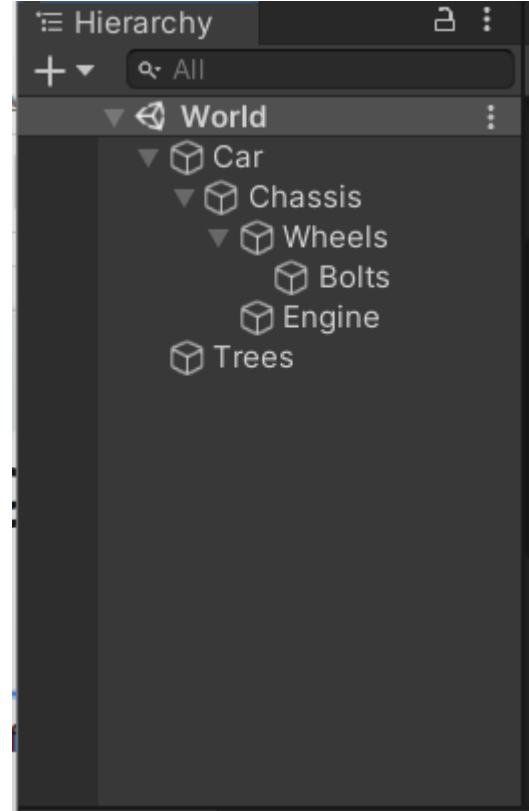
IMPORTANT !! *Changes made to values while in Play Mode are not saved once you stop playing.*



Unity scene graph

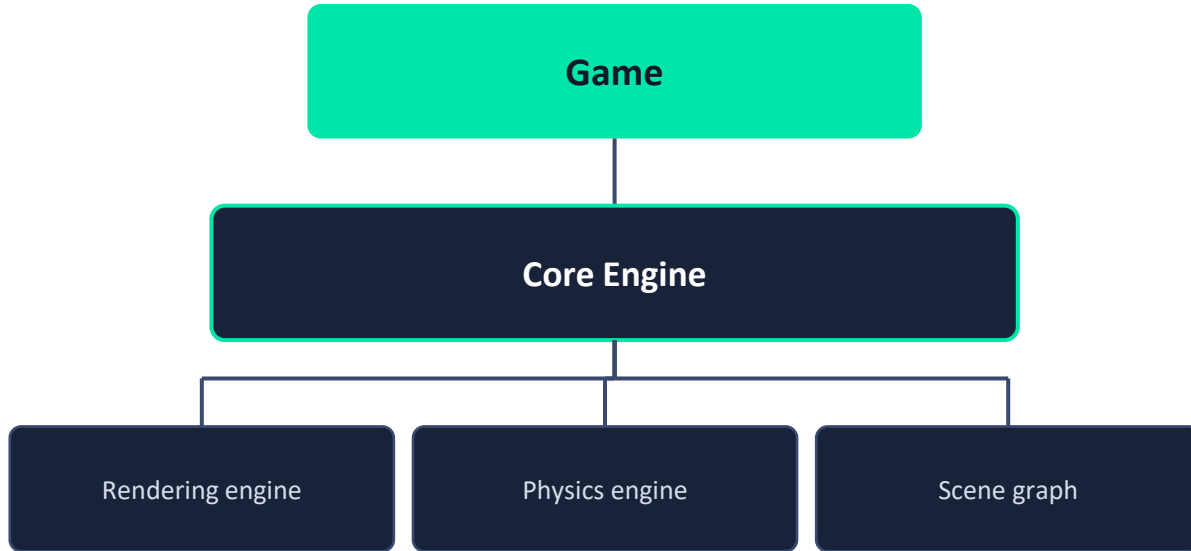
Here is a sample of the scene graph :

- Here we can see a representation of the previous scene graph :



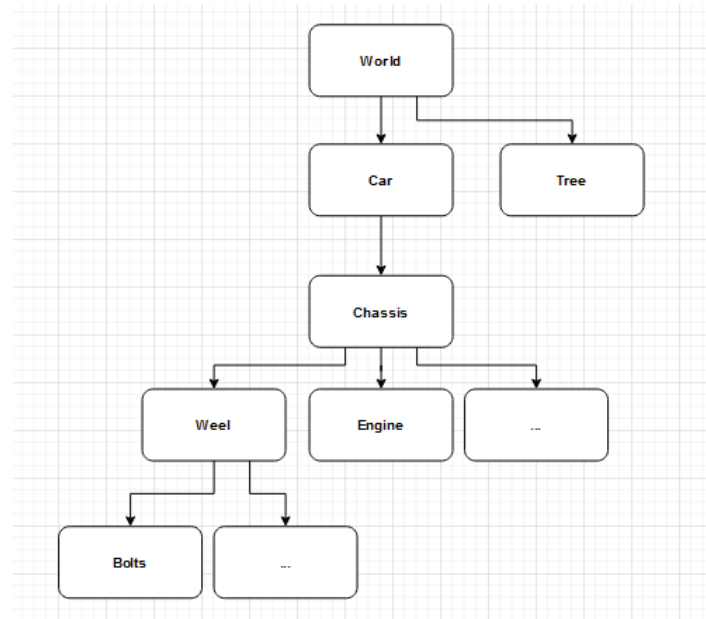
How it all works

A typical game engine structure



What is a scene graph ?

Scene graph is a **structured representation** of a scene that can clearly express the objects, **attributes**, and **relationships between objects in the scene**.





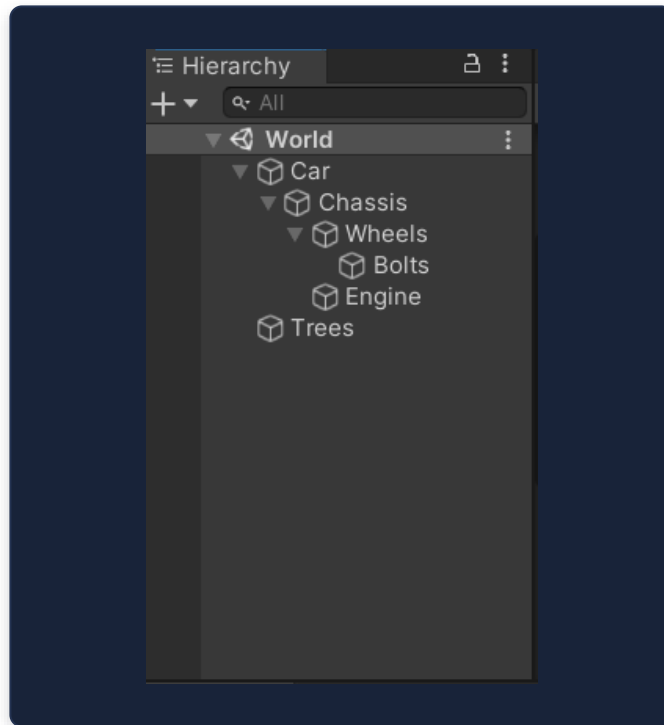
Why use a scene graph ?

- It allows to create a **hierarchy** of objects
- Useful to define **local relationships** between objects instead of using a global relationship
- It allows to **group** objects and **move them relative to each other** (A car doesn't leave it's wheels behind when it drives off)

Unity's Scene: the stage for your game

In Unity, a Scene holds one instance of the scene graph — a menu, a level, or a single room. Games load and unload scenes to move between them.

- Every project has at least one active Scene at a time
- Scenes are saved as .unity files in the Assets folder
- Use `SceneManager.LoadScene()` to switch scenes at runtime
- Large games split content across many small scenes





So what is a game object ?

A node of the scene graph is called a **Game Object** in Unity

GameObjects are the **fundamental objects** in Unity that represent characters, props and scenery.

They do not accomplish much in themselves but they act as containers for **Components**, which implement the real functionality.

Important : All game objects must have a **Transform** component

So what is a GameObject?

A node of the scene graph is called a GameObject in Unity. They act as containers for Components, which implement the real functionality.

Important : Every GameObject must have a Transform.



A named container

Every GameObject has a name, a Transform, and an active/inactive state



Empty by default

It has no behavior until you attach Components to it



A scene graph node

Can be parented to other GameObjects to form a hierarchy

The transform component

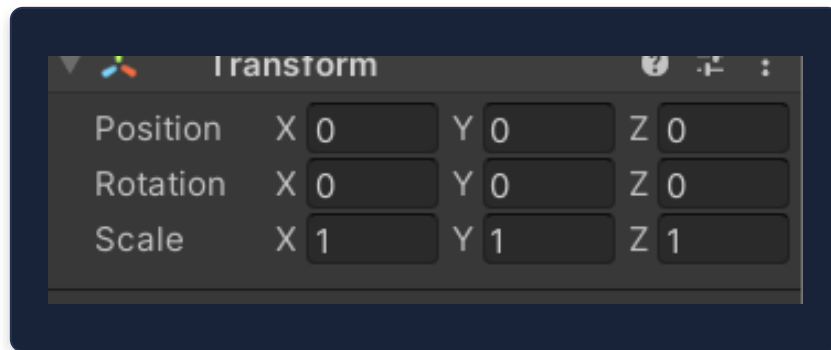
The transform component contains the position, rotation, and scale of an object. Modifying it changes where the object is, how it's oriented, or how big it is — and it also exposes helper vectors like the forward direction.

Position X, Y, Z location in the Scene

Rotation Orientation, stored internally as a quaternion

Scale Size multiplier along each axis

Forward vector Unit vector describing which way the object faces



The Transform component as it appears in the Inspector.



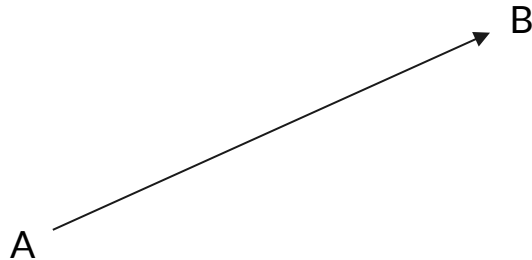
Some Notion of Linear algebra.



Some notion of linear algebra

What is a **vector** ?

A vector is a quantity or phenomenon that has two independent properties: **magnitude** and **direction**. The term also denotes the mathematical or geometrical representation of such a quantity. Examples of vectors in nature are **velocity**, **momentum** and **force**.





Vectors in game development

In game development, vectors are mainly used to describe the **position** of a game object, or to determine its **speed** and **direction**.

In Unity 3D, vectors are an array of 3 values describing 3D coordinates so (x,y,z).

```
Vector3 demoVector = new Vector3(10, 15, 20);
```

In this example, demoVector is defined such as X = 10, Y = 15, Z = 20



Normalizing a vector

When normalized, a vector keeps the same direction but its length is 1.0.

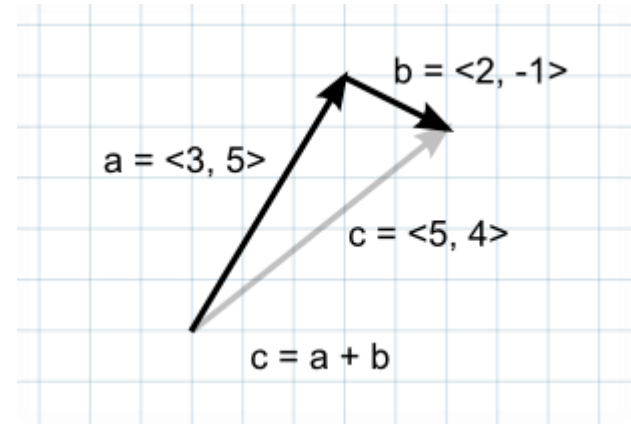
Note that the current vector is unchanged and a new normalized vector is returned. If you want to normalize the current vector, use [Normalize](#) function.

If the vector is too small to be normalized a zero vector will be returned.

```
Vector3 normalizedVector = Vector3.normalize(myVector);
```

Understanding Vector Arithmetic : Addition

When two vectors are added together, the result is equivalent to taking the original vectors as “steps”, one after the other. Note that the order of the two parameters doesn't matter, since the result is the same either way.





Understanding Vector Arithmetic : Addition

If the first vector is taken as a point in space then the second can be interpreted as an offset or “jump” from that position.

For example, to find a point 5 units above a location on the ground, you could use the following calculation:-

```
var pointInAir = pointOnGround + new Vector2(0, 5);
```



Understanding Vector Arithmetic : Addition

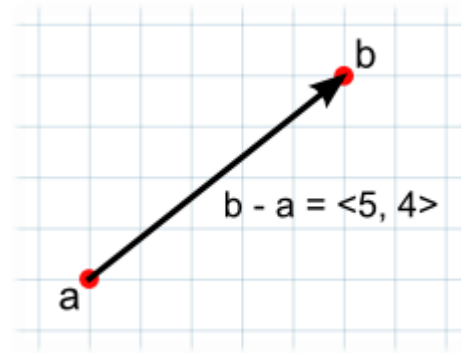
If the vectors represent forces then it is more intuitive to think of them in terms of their direction and magnitude (the magnitude indicates the size of the force).

Adding two force vectors results in a new vector equivalent to the combination of the forces. This concept is often useful when applying forces with several separate components acting at once (eg, a rocket being propelled forward may also be affected by a crosswind).

Understanding Vector Arithmetic : Subtraction

Vector subtraction is **most often used** to get the direction and distance from one object to another.

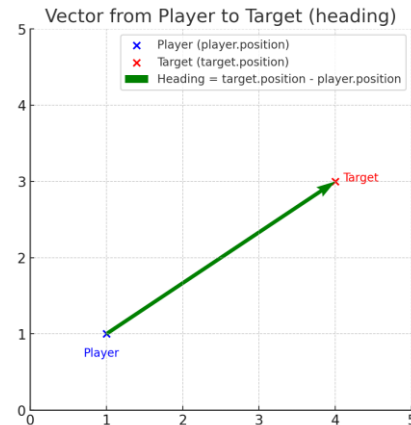
Note that the order of the two parameters does matter with subtraction



Direction and Distance from One point to Another

If one point in space is subtracted from another, then the result is a vector that “points” from one object position to the other:

```
// Gets a vector that points from the player's position to the target's.  
Vector3 fromPlayerToTarget = target.position - player.position;
```





Direction and Distance from One Object to Another

As well as pointing in the direction of the target object, this vector's magnitude is equal to the distance between the two positions.

```
var distanceBetweenPlayerAndTarget = fromPlayerToTarget.magnitude;
```



Direction and Distance from One Object to Another

If you only need to use the distance for comparison (for a proximity check, say) then you can avoid the magnitude calculation altogether.

The `sqrMagnitude` property gives the square of the magnitude value, and is calculated like the magnitude but without the time-consuming square root operation. Rather than compare the magnitude against a known distance, you can compare the squared magnitude against the squared distance.

```
if (heading.sqrMagnitude < maxRange * maxRange) {  
    // Target is within range.  
}
```

Components



Components

Components are the functional pieces of every **GameObject**.

Components contain properties which you can edit to define the behavior of a **GameObject**.

They **can** be added at **runtime** but generally are added before during the edition process.

They are used to be able to customize the behaviour of a specific **GameObject**.



Why do we use a component based architecture ?

Components allow the creation of variation of behaviour **without** the intervention of a programmer. Designers can mix and match already programmed components to create new behaviours for that **GameObject**.

For exemple, a designer could add the **FlamingComponent** to an axe **GameObject** to suddenly have a Flaming Axe in our game.

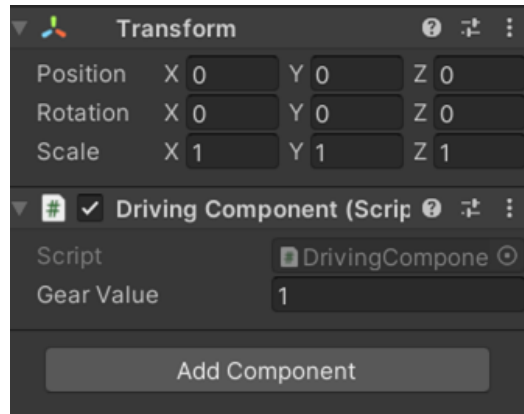
How to program your own components : Monobehaviours

Every **public** variable will show up in the **inspector**, allowing you to modify **those value externally**.

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DrivingComponent : MonoBehaviour
{
    public int GearValue = 1;
    // Start is called before the first frame update
    void Start()
    {
    }

    // Update is called once per frame
    void Update()
    {
    }
}
```





You can also expose private variables with [SerializeField]

```
using UnityEngine;

public class SomePerson : MonoBehaviour
{
    //This field gets serialized because it is public.
    public string firstName = "John";

    //This field does not get serialized because it is private.
    private int age = 40;

    //This field gets serialized even though it is private
    //because it has the SerializeField attribute applied.
    [SerializeField]
    private bool hasHealthPotion = true;

    void Start()
    {
        if (hasHealthPotion)
            Debug.Log("Person's first name: " + firstName + " Person's age: " + age);
    }
}
```



You can exposed custom class too.

```
[System.Serializable]
public class PlayerStats
{
    public int health;
    public int strength;
}
```

```
public class ExposingVariables : MonoBehaviour
{
    [Header("Public Parameters")]
    public string Name;
    public Vector3 SpawnPoint;

    [Header("Public Serialized Parameters")]
    public PlayerStats publicStats;
    public PlayerStats[] publicStatsArray;

    [Header("Private Serialized Parameters with [SerializedField]")]
    [SerializeField] private PlayerStats privateStats;
    [SerializeField] private PlayerStats[] privateStatsArray;

    void Start()
    {
        Debug.Log("Public: " + publicStats);
        Debug.Log("Private: " + privateStats);
    }
}
```



Unity : a component based architecture ?

Generally in computer science , **inheritance** is leveraged within object-oriented programming to **reuse** code without **rewriting** it. In most game engine, however, it's not the approach we use. We use what is called a component based architecture.

Let say we use **inheritance**, and we want to define both ranged and non-ranged weapons. You could say that the crossbow and slingshot inherit some of their functionality from the ranged weapon since they fire projectiles. You could say that an axe is a ranged weapon, too (with a very low range).

Imagine they were hundreds of different weapon types and we want to add or remove a behaviour for some of the weapons. Developers could be stuck maintaining several branches of scripts for a very common behavior used across the entire system.



MonoBehaviours

By default, when you create a new MonoBehaviours Unity will ask you to define 2 methods:

- **Start** : A method that is called on every script to initialize them at the beginning of every game execution. Before the first frame update.
- **Update** : A method that is called once every frame.

The MonoBehaviour lifecycle



Unity calls specific methods on your script automatically, in a predictable order, as the Scene runs.

1

Awake ()

Called once, before the Scene starts, even if disabled

2

Start ()

Called once, right before the first frame

3

Update ()

Called every single frame — the main gameplay loop

4

FixedUpdate ()

Called on a fixed timer — used for physics

Understanding a game engine:

Discrete motion



How Video Games "Trick" Your Brain

A game engine does not actually move objects smoothly. Instead, it runs in a fast loop called the **Game Loop** (often updating 60 times per second, or 60 FPS).

Every time the loop runs:

1. It reads your controller input.
2. It calculates where the object should be.
3. It instantaneously **teleports** the object to the new position.
4. It draws the picture on your screen.

Because this happens so fast, your brain connects the dots and sees continuous, smooth motion—just like how a physical paper flipbook creates the illusion of an animated cartoon.



Moving GameObjects: transform & Translate()

Any MonoBehaviour can access its own GameObject's Transform through the transform shorthand. To move it, you can modify the different vector contained in it. To move the position we would prefer the Translate() method over editing position directly. The translate method simply adds to the current position the vector you use as a parameter

MoveExample.cs

```
void Update()
{
    transform.Translate(Vector3.forward * speed * Time.deltaTime);
    // equivalent to using transform.position += Vector3.forward * speed * Time.deltaTime
}
```

Discrete motion: why Time.deltaTime?

A game doesn't move continuously — it updates in discrete steps, one frame at a time. Each Update() call nudges every object forward a little, then the engine draws the result and moves to the next frame.

WITHOUT DELTATIME

```
transform.Translate(Vector3.forward * speed);
```

Moves the same distance every frame — so a 30 fps machine moves half as fast as a 60 fps one.

WITH TIME.DELTATIME

```
transform.Translate(Vector3.forward * speed *  
Time.deltaTime);
```

Scales movement by the real time elapsed — the same speed no matter the frame rate.



Unity Important class : Time

Unity's Time class provides important basic properties that allow you to work with time-related values in your project.

The Time class has a few properties which provide you with numeric values that allow you to measure time elapsing while your game or app is running. For example:

Time.time returns the amount of time in seconds since your project started playing.

Time.deltaTime returns the amount of time in seconds that elapsed since the last frame completed. This value varies depending on the **frames per second** (FPS) rate at which your game or app is running



Unity Important class : Time

The Time class also provides you with properties which allow you to control and limit how time elapses, for example:

Time.timeScale controls the rate at which time elapses. You can read this value, or set it to control how fast time passes, allowing you to create slow-motion effects.

Time.fixedDeltaTime controls the interval of Unity's **fixed timestep** loop (used for physics, and if you want to write deterministic time-based code).

Time.maximumDeltaTime sets an upper limit on the amount of time the engine will report as having passed by the "delta time" properties above.



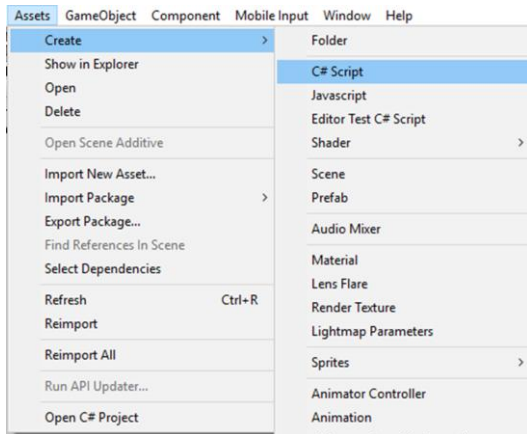
MonoBehaviours

- You can access the transform of a gameobject that contains your MonoBehaviour by using the “transform” shorthand.
- To move a transform it is advised to use the translate method.

Adding new behaviour (components) to your GameObject

Not all behaviour for our game already have existing components for the job. To add a new component we have to create a **script** that defines it's **behaviour**.

To add a new behaviour we need to create a new class that inherits a class **MonoBehaviour** in Unity



```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

public class DrivingComponent : MonoBehaviour
{
    public int GearValue = 1;
    // Start is called before the first frame update
    void Start()
    {

    }

    // Update is called once per frame
    void Update()
    {

    }
}
```



MonoBehaviour : Translate example

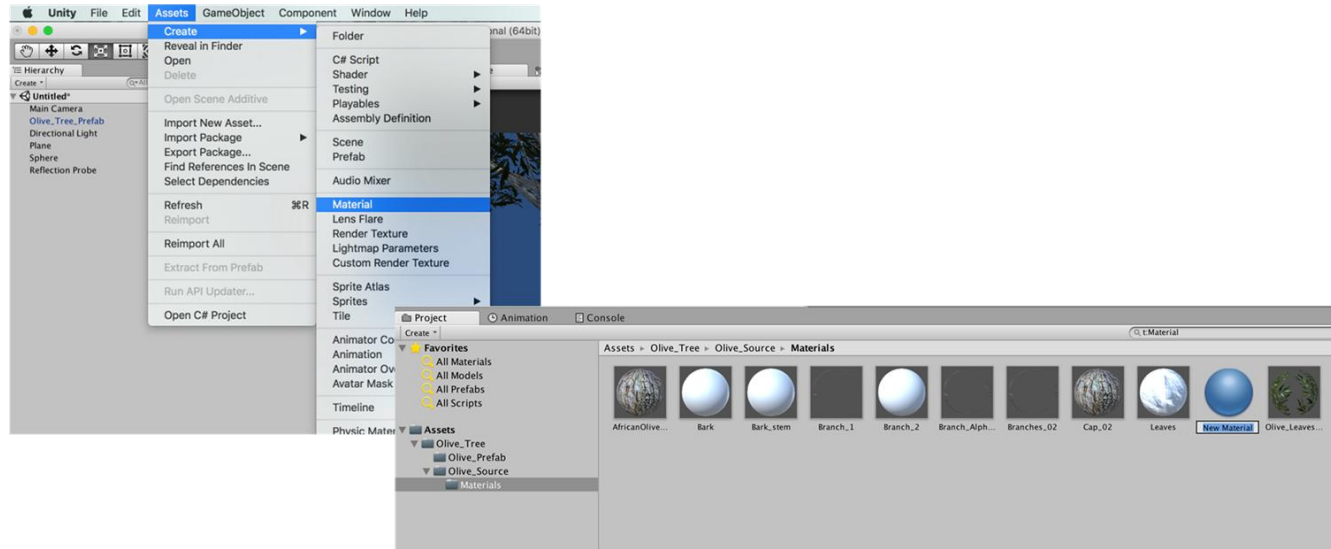
```
void Update()
{
    if (Input.GetKey(KeyCode.W))
    {
        transform.Translate(Vector3.up * Time.deltaTime);
        print("W key is held down");
    }

    if (Input.GetKey(KeyCode.S))
    {
        transform.Translate(Vector3.down * Time.deltaTime);
        print("S key is held down");
    }
}
```

Important classes

Unity's got you covered

Many common behaviour have a component that already exist in Unity





Unity API : Camera component

Cameras are the devices that capture and display the world to the player. By customizing and manipulating cameras, you can make the presentation of your game truly unique. You can have an unlimited number of cameras in a **scene**

They can be set to render in any order, at any place on the screen, or only certain parts of the screen.

But without a camera your game will have nothing to display. So you need to have **at least one** camera in your scene.

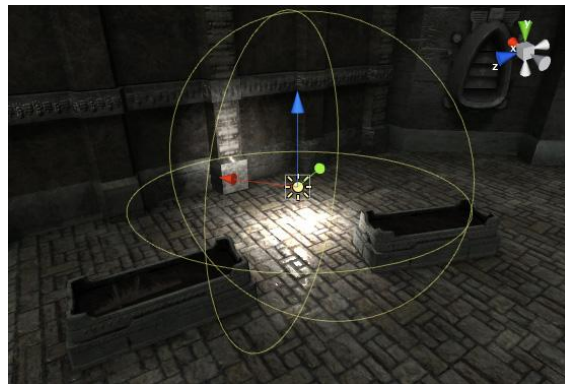
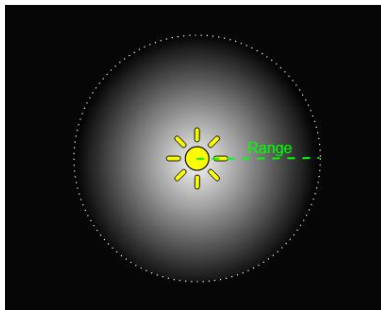
Unity API: Point lights

To be able to see in our game we use several types of lights to approximate lighting in the real world

A **Point Light** is located at a point in space and sends light out in all directions **equally**. The intensity diminishes with **distance** from the light, reaching zero at a specified range.

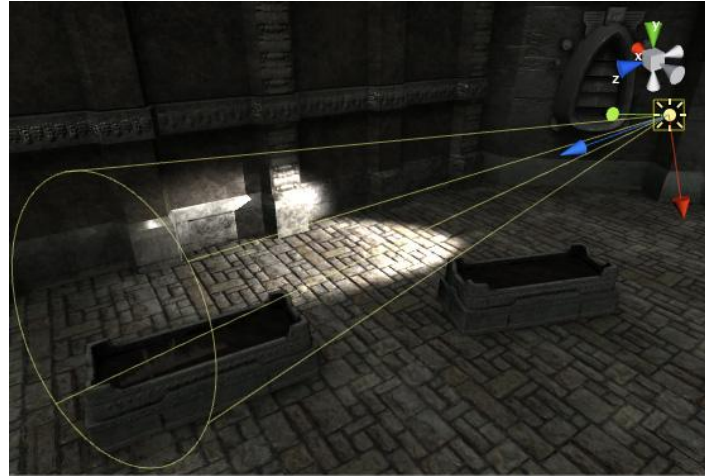
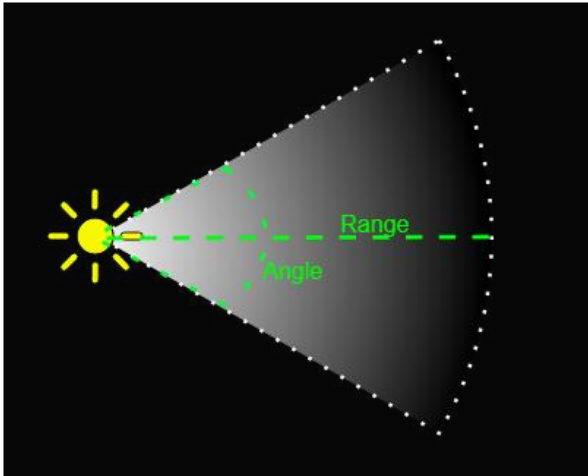
Point lights are useful for simulating lamps and other local sources of light in a scene.

You can also use them to make a spark illuminate its surroundings.



Unity API: Spot Lights

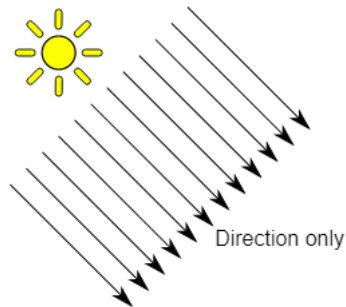
Like a Point Light, a **Spot Light** has a specified **location** and **range** over which the light falls off. However, a Spot Light is **constrained** to an **angle**, resulting in a **cone-shaped** region of illumination.



Unity API: Directional Lights

Directional Lights are useful for creating effects such as **sunlight** in your scenes. **Behaving** in many ways **like the sun**, directional lights can be thought of as **distant light sources** which exist very far away.

A Directional Light **doesn't have** any identifiable **source position** and so the light object **can be placed** **anywhere in the scene**. **All** objects in the scene are illuminated **as if the light is always from the same direction**. The **distance** of the light from the target object **isn't defined** and so the light **doesn't diminish**.





Unity API : Meshes and Materials

To draw something in Unity, you must provide information that describes its shape, and information that describes the appearance of its surface.

You use **meshes** to describe **shapes**, and **materials** to describe the **appearance of surfaces**.

Materials are used in conjunction with **textures** to create complex artwork.



Unity API : Input

Interface into the Input system.

Use this class to read the axes set up in the Conventional Game Input, and to access multi-touch/accelerometer data on mobile devices.

The Input class has several methods to access the keys that are pressed as well as gamepad input.

Input flags **are not** reset until the next **Update**. You should make **all** the **Input** calls in the **Update Loop**.



Unity API : Input

How to read inputs :

`Input.GetKeyDown(KeyCode.Space)`

will return true if the specified Keycode is **has been pressed down in the last frame**

`bool value = Input.GetKeyUp(KeyCode.Space)`

will return true if the specified Keycode **has been release in the last frame.**

`bool value = Input.GetKey(KeyCode.Space)`

will return true if the specified Keycode **is being currently pressed down.**



Unity API : Input

- `Input.GetAxis("Vertical")`
will return the value of the vertical axis of the gamepad from (-1 to 1).
- `Input.GetAxis("Horizontal")`
will return value of the horizontal axis of the gamepad from (-1 to 1).

Unity API : Input

```
bool value = Input.GetButtonDown("Fire1")
```

will return true if the button mapped to Fire1 has been **pressed down in the last frame**

```
bool value = Input.GetButtonUp("Fire1")
```

will return true if the button mapped to Fire1 **has been release in the last frame.**

```
bool value = Input.GetButton("Fire1")
```

will return true if the button mapped to Fire1 **is currently being pressed down.**



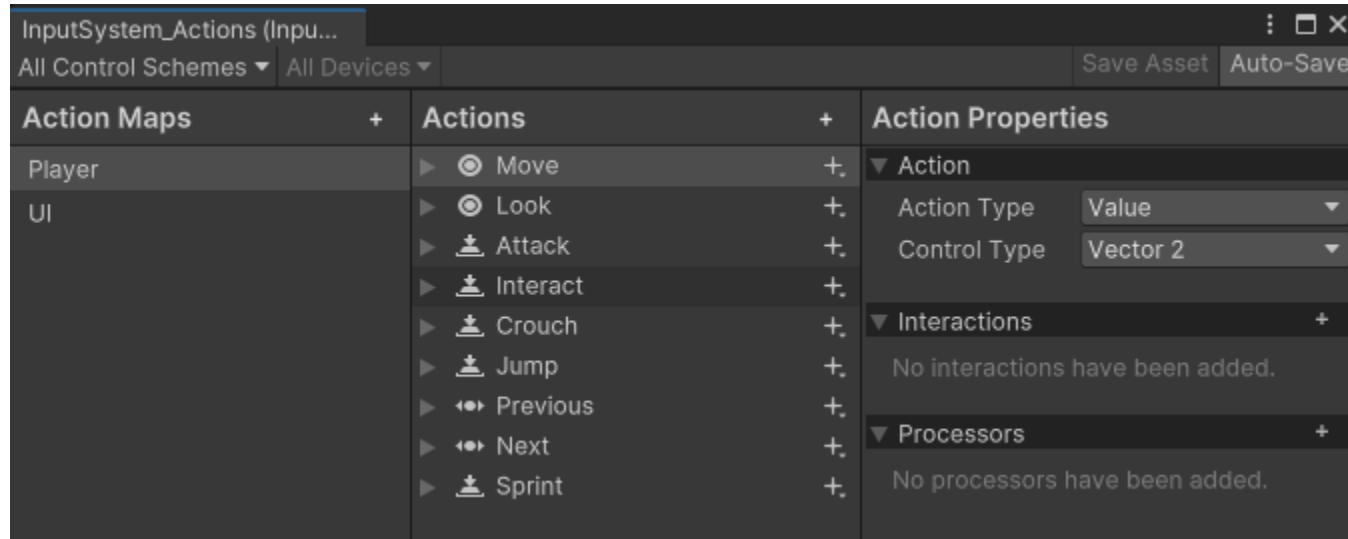


Unity API: Input (coding examples)

```
// Update is called once per frame
void Update()
{
    if (Input.GetKeyDown(KeyCode.W))
    {
        print("up arrow key is held down");
    }

    if (Input.GetKeyDown(KeyCode.S))
    {
        print("down arrow key is held down");
    }
}
```


Input system : Newer and more flexible system



The new Input System: Input Actions

Newer Unity projects use the Input System package instead of the Input Manager. Input is captured through Input Actions — reusable, named actions mapped to any device via an Input Actions asset.



Action Maps

Group related actions together — a "Player" map for gameplay, a separate "UI" map for menus.



Bindings

Each action can bind to multiple controls at once, so keyboard, gamepad, and touch all trigger the same action.



Control Schemes

Define which device types belong together, so Unity can auto-switch between keyboard/mouse and gamepad.

Input Actions
Asset

Action Map
"Player"

Action
"Move"

Binding
W A S D / Stick

Reading input via Actions

A PlayerInput component connects an Input Actions asset to your script. Unity calls a matching method for each action automatically.

- Attach PlayerInput and assign an Input Actions asset
- Unity auto-calls On<ActionName> — e.g. OnMove, OnJump
- Or subscribe directly to an action's performed event
- Generate a typed C# class to avoid string lookups

<https://docs.unity3d.com/Packages/com.unity.inputsystem@1.0/manual/QuickStartGuide.html>

```
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerInputHandler : MonoBehaviour {
    private InputAction moveAction;
    private InputAction jumpAction;

    private Vector2 move;
    private bool jump;

    void Start() {
        moveAction = InputSystem.actions["Move"];
        jumpAction = InputSystem.actions["Jump"];
    }

    void Update() {
        move = moveAction.ReadValue<Vector2>();
        jump = jumpAction.WasPressedThisFrame();
    }
}
```